

AI AND IOT SOLUTIONS FOR FOOD FRESHNESS & SAFETY

PROJECT REPORT

Submitted by

LAVANYAH B [Reg No:RA2111003011181]

KIRTHIC MADAVAN S [Reg No:RA2111003011196]

RAGHUL SIDDARATH C [Reg No:RA2111003011209]

Under the Guidance of

Dr.ANBAZHAGU U.V

Assistant Professor, Department of Computing Technologies

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING



**DEPARTMENT OF COMPUTING TECHNOLOGIES
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR- 603 203**

MAY 2025



Department of Computing Technologies
SRM Institute of Science & Technology
Own Work Declaration Form

Degree/ Course : B. TECH IN COMPUTER SCIENCE AND ENGINEERING
Student Name : LAVANYAH B, KIRTHIC S & RAGHUL SIDDARTH C
Registration Number : RA2111003011181, RA2111003011196 & RA2111003011209
Title of Work : AI AND IOT SOLUTIONS FOR FOOD FRESHNESS & SAFETY

We here by certify that this assessment compiles with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guideline.

We confirm that all the work contained in this assessment is my / our own except where indicated, and that I / We have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g. fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook / University website

We understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:

We are aware of and understand the University's policy on Academic misconduct and plagiarism and we certify that this assessment is our own work, except where indicated by referring, and that we have followed the good academic practices noted above.

Student 1 Signature:

Student 2 Signature:

Student 3 Signature:

Date:

If you are working in a group, please write your registration numbers and sign with the date for every student in your group.



SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 18CSP109L - Major Project report titled “**AI AND IOT SOLUTIONS FOR FOOD FRESHNESS & SAFETY** ” is the Bonafide work of **LAVANYAH B [RegNo:RA2111003011181]**, **KIRTHIC MADAVAN SUNDARAVEL [RegNo:RA2111003011196]** and **RAGHUL SIDDARTH CHANDRASEKAR [RegNo:RA2111003011209]** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Dr.U.V.ANBAZHAGU
SUPERVISOR
Assistant Professor
Department of Computing Technologies

Dr.G.NIRANJANA
HEAD OF THE DEPARTMENT & PROFESSOR
Department of Computing Technologies

EXAMINER 1

EXAMINER 2

ACKNOWLEDGEMENTS

We express our humble gratitude to **Dr.C.Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to **Dr.Leenus Jesu Martin M**, Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank **Dr.Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to, **Dr.M.Pushpalatha**, Professor and Associate Chairperson, School of Computing and **Dr.C.Lakshmi**, Professor and Associate Chairperson, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, **Dr.G.Niranjana**, Professor, Department of Computing Technologies, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We want to convey our thanks to our Project Coordinators, **Dr.R.Vidhya** Assistant Professor, **Dr.M.Arul Prakash**, Assistant Professor, **Dr.M.Revathi**, Assistant Professor, Panel Head, **Dr.R.Srinivasan**, Associate Professor and Panel Members, **Dr.A.Arul Murugan**, Assistant Professor, **Dr.R.Hariharan**, Assistant Professor, Department of Computing Technologies, SRM Institute of Science and Technology, for their inputs during the project reviews and support.

We register our immeasurable thanks to our Faculty Advisor, **Dr.U.V.Anbazhagu**, Assistant Professor, Department of Computing Technologies, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, **Dr.U.V.Anbazhagu**, Assistant Professor, Department of Computing Technologies, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under her mentorship. She provided us with the freedom and support to explore the research topics of our interest. Her passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank all the staff and students of Computing Technologies, School of Computing, S.R.M Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement.

LAVANYAH B [Reg No: RA2111003011181]

KIRTHIC MADAVAN S [Reg No: RA2111003011196]

RAGHUL SIDDARATH C [Reg No: RA2111003011209]

ABSTRACT

The integration of Artificial Intelligence (AI) and the Internet of Things (IoT) introduces an innovative solution for enhancing quality control throughout the food supply chain, particularly for perishable goods like fruits, vegetables, and meats. Addressing the critical need for precise storage and transport conditions, this project focuses on the creation of a smart, automated monitoring system. It utilizes a network of IoT sensors to continuously monitor vital environmental factors—such as temperature, humidity, and gas levels—that influence the freshness and longevity of food products. These sensor readings are transmitted in real time to cloud-based infrastructure, where a Convolutional Neural Network (CNN) model analyzes the data to detect early spoilage indicators, assess contamination risks, and classify food freshness levels. The system includes a dynamic, cloud-hosted dashboard that presents sensor metrics and AI-driven predictions in a user-friendly interface. This platform offers live alerts, detailed analytics, and access to historical data trends, enabling users to take timely action. By combining AI and IoT, the system enhances decision-making across the supply chain, reduces food waste, safeguards public health, and improves overall efficiency. Built using an agile development approach with iterative improvements, the system emphasizes high accuracy, fast response times, and dependable performance. Additionally, the project aligns with key United Nations Sustainable Development Goals (SDGs), including Zero Hunger, Good Health and Well-being, and Responsible Consumption and Production, by supporting smarter food handling and promoting sustainability across the supply network.

TABLE OF CONTENTS

ABSTRACT		vi
TABLE OF CONTENTS		vii
LIST OF FIGURES		ix
LIST OF TABLES		x
ABBREVIATIONS		xi
CHAPTER NO.	TITLE	PAGE NO.
1	INTRODUCTION	1
	1.1 Introduction	2
	1.2 Problem Statement	2
	1.3 Objective	3
	1.4 Scope of the Project	3
	1.5 Relevance to SDGs	4
	1.6 Methodology Overview	5
	1.7 Report Structure	6
2	LITERATURE SURVEY	6
	2.1 Review of AI in Quality Control	6
	2.2 Use of IoT in Supply Chain Monitoring	7
	2.3 CNN for Image Classification	8
	2.4 Transfer Learning for Freshness Detection	9
	2.5 Summary of Research Gaps	9
3	SPRINT PLANNING AND EXECTION METHODOLOGY	10
	3.1 SPRINT I	11
	3.1.1 Objectives with user stories of Sprint I	12
	3.1.2 Functional Requirements	14
	3.1.3 System Architecture	16
	3.1.4 Sprint Outcome	17
	3.1.5 Sprint Retrospective	19
	3.2 SPRINT II	20
	3.2.1 Objectives and Enhancement	20

3.2.2 Updated Architecture	21
3.2.3 Functional Modules	22
3.2.4 Performance Metrics	23
3.2.5 Sprint Retrospective	24
4 SYSTEM INTEGRATION AND TESTING	25
4.1 Integration Approach	25
4.2 System Validation and Testing Strategy	25
4.3 Observations and Outcome	26
5 USER INTERFACE DESIGN AND DASHBOARD FEATURES	27
5.1 Dashboard Design Principles	27
5.2 Dashboard Features and Components	28
6 RESULTS AND DISCUSSIONS	29
6.1 Classification Model Performance	29
6.2 Sensor Network Evaluation	29
6.3 Real-Time Monitoring and Alert Accuracy	30
6.4 Dashboard Usability and User Feedback	30
6.5 Comparative Analysis with Traditional Methods	30
6.6 Summary of Findings	30
7 CONCLUSION AND FUTURE ENHANCEMENT	31
REFERENCES	32
APPENDIX	33
A CODING	34
B PLAGIARISM REPORT	54
C PUBLICATION	55

LIST OF FIGURES

CHAPTER NO.	TITLE	PAGE NO.
3.1	IoT Sensor Network Setup (ESPS266 + Sensors)	14
3.2	System Architecture Diagram	16
3.3	Architecture of Shufflenet	18
3.4	Architecture of SqueezeNet	21
3.5	Activity Diagram	22
5.1	Hardware Block Diagram	27

LIST OF TABLES

CHAPTER NO.	TITLE	PAGE NO.
3.1	CNN Layer Configuration	24
6.1	Performance Comparison	30

ABBREVIATIONS

SPP	Spatial Pyramid Pooling
FIR	Finite Impulse Response
SSDAE	Stacked Sparse Denoising Autoencoder
MQTT	Message Queuing Telemetry Transport
LCD	Liquid Crystal Display
OLED	Organic Light Emitting Diode
ESP	Espressif Systems
IDE	Integrated Development Environment
DFD	Data Flow Diagram
UML	Unified Modeling Language
SMOTE	Synthetic Minority Oversampling Technique
ROC	Receiver Operating Characteristic Curve

CHAPTER 1

INTRODUCTION

1.1 Introduction

The swift expansion of the global food industry has highlighted the urgent need for better monitoring and management of perishable food products like fruits, vegetables, and meat. These items are crucial to daily nutrition but are extremely vulnerable to environmental factors. Minor changes in temperature, humidity, or storage time can result in spoilage, contamination, and potential health risks. Conventional methods of assessing quality typically depend on human inspection and manual grading, which often suffer from inconsistencies, subjective judgment, and time delays.

In recent times, the integration of Artificial Intelligence (AI) and the Internet of Things (IoT) has opened new avenues for transforming the food supply chain. AI, particularly through deep learning methods, enables automated analysis and prediction of food freshness using data from images and sensors. At the same time, IoT technology supports the real-time monitoring and transmission of environmental conditions using sensors positioned throughout the supply network—from agricultural fields and storage center to transportation and retail locations.

This project harnesses the power of both AI and IoT to develop a smart, unified system that actively tracks the condition of perishable food items. Its goal is to deliver continuous freshness assessment, contamination detection, and environmental monitoring to maintain ideal storage and handling conditions. By streamlining the quality assurance process, the system helps reduce reliance on manual checks, lower the risk of human error, decrease food spoilage, and ultimately promote food safety while boosting consumer trust.

1.2 Problem Statement

A significant challenge in today's food industry is maintaining consistent and accurate monitoring of perishable goods throughout the stages of post-harvest handling, storage, and transport. Even with progress in logistics and improvements in cold storage systems, spoilage of food remains a common issue. This ongoing problem results in major economic losses, contributes to food scarcity, and increases the risk of health issues from consuming tainted or spoiled products.

The core issue lies in the lack of real-time and intelligent quality monitoring mechanisms that can operate autonomously and deliver accurate predictions. Manual inspection processes are labor-intensive and often ineffective in large-scale operations. Furthermore, traditional systems do not possess the predictive capability to prevent spoilage or identify early signs of degradation. Existing quality assurance practices are reactive rather than proactive, meaning action is often taken only after spoilage has occurred.

Given the limitations of current practices, there is a growing demand for a scalable, automated solution that not only monitors freshness conditions but also predicts potential risks and ensures compliance with food safety standards. This project directly addresses this need by combining AI-powered image classification with IoT-based environmental sensing to form a unified monitoring platform for food quality assurance.

1.3 Objective

The central aim of this project is to create an advanced, AI and IoT-driven system for ensuring the quality of perishable food products. This system is designed to assess freshness levels, identify environmental irregularities, and issue real-time alerts to help prevent spoilage.

Specifically, the project seeks to build an intelligent, unified platform that leverages Artificial Intelligence (AI) and the Internet of Things (IoT) to track and preserve the freshness and safety of items like fruits, vegetables, and meat. To accomplish this, various IoT sensors—such as those measuring temperature, humidity, and gas levels—are strategically deployed throughout the supply chain to continuously capture vital environmental data.

This constant flow of real-time information serves as the foundation for a proactive approach to food quality management. In parallel, a Convolutional Neural Network (CNN) is developed and trained to evaluate food images by identifying signs of freshness or spoilage, allowing for automated and precise assessments without the need for manual checks. The AI predictions and sensor data are then synchronized and displayed via a cloud-based dashboard, offering supply chain stakeholders instant access to storage conditions and product quality from any location.

To ensure timely interventions, the system is configured to generate automated alerts and notifications whenever environmental conditions breach predefined safety thresholds. By facilitating early detection of spoilage risks and enabling corrective actions, the system aims to significantly reduce food wastage, enhance consumer safety, and promote sustainable food management practices.

Collectively, these objectives aim to enhance the reliability, efficiency, and responsiveness of food quality monitoring systems, making the supply chain smarter, safer, and more resilient to potential risks. This system aims to combine real-time environmental monitoring through sensors with predictive analysis using Convolutional Neural Networks (CNN) to classify food quality based on visual indicators. It also seeks to implement a cloud-based dashboard for remote access, providing stakeholders with real-time insights into product conditions. By automating freshness detection and environmental tracking, the project aims to reduce food spoilage, improve safety standards, and support data-driven decision-making in the food supply chain. Additional goals include developing scalable modules that can be expanded to other food categories, incorporating transfer learning for higher accuracy with limited data, and enabling automated alerts to ensure timely intervention.

1.4 Scope of the Project

As far as this project is concerned, its focus is on the creation and deployment of an AI-IoT system for monitoring the freshness of fruits, vegetables, and even meat. The system is intended to be modular, scalable, and flexible for application for different areas in the food industry such as agriculture, warehousing, cold storage, and retail. The major components of the project include:

- **Hardware:** IoT-based environmental sensors (DHT11 for temperature/humidity, MQ135 for gas detection), ESP8266 microcontroller, and camera modules for image acquisition.
- **Software:** CNN-based classification models built using Python and TensorFlow; cloud integration using Adafruit IO; a responsive dashboard for data visualization.
- **Dataset:** A curated image dataset of food items categorized by freshness level; real-time data from sensor modules.
- **Deployment Environment:** Simulated cold storage setup to validate system performance, with potential for real-world application and scalability to broader supply chains.

The project focuses on a proof-of-concept prototype but lays the foundation for future enhancements such as mobile application access, integration with blockchain for traceability, and extended support for diverse food categories.

1.5 Relevance to Sustainable Development Goals (SDGs)

This project aligns with several of the United Nations Sustainable Development Goals (SDGs), addressing issues related to food security, public health, and environmental sustainability:

The proposed system strongly aligns with several United Nations Sustainable Development Goals (SDGs). In support of **SDG 2: Zero Hunger**, the system helps reduce post-harvest losses by enabling early detection of spoilage and facilitating more efficient food distribution, thereby improving food availability and accessibility. Addressing **SDG 3: Good Health and Well-being**, the system minimizes the consumption of spoiled or contaminated food, thus promoting public health and reducing the incidence of foodborne illnesses.

Furthermore, the project contributes to **SDG 12: Responsible Consumption and Production** by encouraging smart monitoring practices that lead to reduced waste and enhanced sustainability across the food supply chain. Through real-time tracking and intelligent intervention mechanisms, the system promotes more responsible and efficient food production and distribution practices. In essence, this project acts as a technological enabler of sustainable food systems, empowering stakeholders to achieve global sustainability targets through innovation and smart resource management.

1.6 Methodology Overview

This project adopts an agile, sprint-based development methodology to facilitate iterative progress, frequent testing, and early-stage validation of functionality. The agile framework ensures flexibility and adaptability throughout the development process, allowing modifications based on feedback and evolving requirements. The entire development cycle is organized into two major sprints. **Sprint I** concentrated on building the foundational infrastructure of the system, which included collecting image and sensor data, setting up and calibrating IoT hardware, and developing an initial version of the Convolutional Neural Network (CNN) model.

This phase also included testing of the individual modules and establishing basic connectivity between hardware components and the cloud. **Sprint II**, on the other hand, emphasized system-level optimization and expansion. During this sprint, the CNN model was refined using augmented and balanced datasets to improve classification accuracy, while the dashboard was enhanced for better user interaction and real-time alert delivery. Full system integration was achieved, combining the AI model, IoT sensor network, and cloud platform into a unified solution.

The methodological phases include several critical tasks. **Data collection** involved acquiring a diverse set of labelled images representing food at various freshness stages, along with real-time data from temperature, humidity, and gas sensors. **Preprocessing** steps such as image resizing, normalization, and augmentation were conducted to enhance model generalizability, while sensor calibration ensured reliable environmental readings.

In the **model development** phase, a CNN was designed and trained using Python and TensorFlow frameworks, with transfer learning techniques applied to overcome dataset limitations. The **system integration** process linked the ESP8266 microcontroller with the sensors and cloud server via MQTT protocol, enabling seamless real-time data transfer. **Testing and evaluation** included measuring system performance using metrics like accuracy, precision, recall, latency, and response time.

1.7 Report Structure

The structure of this report has been carefully designed to present a clear and logical flow of information, providing a comprehensive view of the project's motivation, technological foundation, implementation process, and outcomes.

Chapter 1 – Introduction outlines the context and motivation behind the project. It includes the background of the study, a well-defined problem statement, the objectives, scope, relevance to the United Nations Sustainable Development Goals (SDGs), and an overview of the development methodology.

Chapter 2 – Literature Survey presents a review of existing research in the domain of food quality monitoring using AI and IoT. It highlights the strengths and limitations of previous studies, identifies research gaps, and forms the basis for defining the project's research objectives and planned methodology. **Chapter 3 – Sprint Planning and Execution** elaborates on the agile sprint-based development strategy employed in this project. It describes the technical architecture, functional modules, objectives, challenges, and outcomes of Sprint I and Sprint II, along with retrospectives and lessons learned.

Chapter 4 – Results and Discussion provides an in-depth analysis of the experimental outcomes. It details the performance of the CNN model using metrics such as accuracy, recall, and F1-score, examines the stability and responsiveness of the sensor network, and compares the proposed solution with conventional quality control systems.

Chapter 5 – Conclusion and Future Enhancements summarizes the overall achievements of the project and reflects on limitations. It proposes future directions, such as mobile app integration, extension to a wider range of food products, and advanced predictive capabilities using spectral imaging.

The final sections, **References and Appendices**, include the complete list of cited works and supplementary content such as Python source code, Arduino sketches, sensor circuit diagrams, screenshots of the dashboard, certificates from conferences, and the plagiarism report. Together, these components provide a holistic understanding of the project, its contributions, and its potential for real-world impact.

CHAPTER 2

LITERATURE SURVEY

2.1 Review of AI in Quality Control

Artificial Intelligence (AI) has emerged as a transformative force in modern food quality control systems. It enables machines to mimic human-level perception in detecting visual cues associated with freshness, contamination, and spoilage in food products. Machine learning algorithms, especially those based on deep learning, offer scalable and accurate alternatives to traditional, manual inspection methods that are often subjective and labor-intensive.

Convolutional Neural Networks (CNNs) have shown remarkable success in tasks involving food quality detection and classification. In [1], researchers proposed a CNN model to assess fruit freshness by analyzing images, achieving high accuracy and reliability in comparison to conventional grading methods. Another study [2] focused on AI-enabled grading of fruits for customer-facing applications, where features like color, shape, and texture were used to predict ripeness and shelf life.

AI also supports predictive maintenance in supply chains, where models can forecast deterioration or recommend preventive action before spoilage occurs. These capabilities significantly reduce food waste, improve safety standards, and ensure consumer satisfaction [3].

2.2 Use of IoT in Supply Chain Monitoring

The Internet of Things represents an underlying technology when it comes to modernizing food supply chains especially when dealing with monitoring and maintaining perishable items. An IoT system allows for real-time monitoring and control of different industrial IoT parameters and physically located in storage houses, transport boxes, and shops where sensors can be mounted on containers or retailing space, there is a full set of storage fleets on transport boxes that enable Interconnected Perishable Goods to communicate with each other. As discussed in [4], the integration of IoT-enabled cold storage units with cloud-based platforms provides a robust mechanism for early detection of unfavorable conditions that could compromise food quality. Sensors like **DHT11** (for temperature and humidity) and **MQ135** (for air quality and gas detection) are cost-effective yet reliable tools that gather environmental data continuously. This data, when fed into AI-driven analytics platforms, enables predictive modeling that anticipates spoilage risks before they manifest physically [5].

Moreover, the communication between these IoT devices and cloud infrastructure is often facilitated through lightweight and efficient messaging protocols such as **MQTT (Message Queuing Telemetry Transport)**. This allows for seamless, low-latency data transmission across long distances, making it ideal for real-time applications [6,7]. Data collected from multiple sensor nodes can be logged, visualized, and analyzed remotely via dashboards, enabling supply chain managers and quality control personnel to make informed decisions instantly.

The implementation of IoT in supply chain monitoring not only enhances traceability and transparency but also significantly reduces food waste by ensuring timely interventions. By automating data collection and enabling intelligent decision-making, IoT contributes to building a smarter, more responsive, and sustainable food supply system.

2.3 CNN for Image Classification

Convolutional Neural Networks (CNNs) have emerged as a powerful and reliable deep learning architecture for image classification tasks, making them particularly suitable for food quality monitoring applications. CNNs are capable of automatically extracting and learning hierarchical features—such as edges, textures, shapes, and color patterns—from raw images without the need for manual feature engineering. This automated feature extraction allows CNNs to effectively distinguish between subtle variations in the visual appearance of food items, including discoloration, fungal growth, shrinkage, or bruising, which are early indicators of spoilage or contamination.

In the context of perishable food assessment, CNNs have demonstrated exceptional performance in classifying the freshness and edibility of items such as fruits, vegetables, and meat. A notable study in [8] implemented a VGG-based CNN model to analyze images of fruits at various stages of freshness. The model achieved an accuracy exceeding 95%, clearly demonstrating the effectiveness of CNNs in distinguishing between fresh and spoiled items based on surface characteristics.

Another research work presented in [9] explored the use of advanced CNN architectures such as ResNet and MobileNet to evaluate minute changes in fruit skin texture and coloration. These models offered high precision in freshness classification and proved efficient in deployment scenarios with limited computational resources. One of the significant advantages of CNNs is their flexibility and adaptability. They can be fine-tuned or retrained using transfer learning to suit a wide range of food categories and environmental conditions, including different lighting, backgrounds, and camera angles. This modularity makes CNN-based systems highly scalable and capable of handling diverse datasets.

CNNs can also be optimized for real-time performance, making them an ideal choice for integration with IoT-based monitoring systems in cold storage units, retail displays, and food inspection lines. Additionally, CNNs can be extended to perform multi-class classification, enabling them to assign quality labels such as “fresh,” “slightly spoiled,” or “rotten” rather than binary outcomes. This provides stakeholders with more granular information for decision-making. With their robust image analysis capabilities and compatibility with low-cost hardware platforms, CNNs play a vital role in enabling intelligent, automated, and scalable food quality inspection systems [10].

2.4 Transfer Learning for Freshness Detection

Transfer learning is a widely used technique in deep learning that allows a pre-trained model, originally developed for a generic task, to be repurposed for a more specific but related application. This is particularly beneficial in fields like agriculture and food quality monitoring, where assembling large, well-labeled image datasets is challenging due to seasonal availability, resource limitations, and the perishable nature of the subjects. Transfer learning helps bridge the gap between data scarcity and model accuracy by enabling knowledge gained from one domain to be applied in another with minimal additional training.

In the case of food freshness detection, transfer learning has proven to be a highly effective strategy. It leverages models trained on vast image datasets, such as ImageNet, to perform classification tasks with relatively few food-specific images. For instance, a study in [11] utilized the MobileNet architecture and applied transfer learning techniques to detect the freshness of vegetables.

The pre-trained network was fine-tuned using a small but targeted dataset, resulting in a highly accurate and computationally efficient model. The use of MobileNet also ensured that the model could be deployed on lightweight, embedded devices, making it suitable for IoT-based food monitoring systems.

Another significant application of transfer learning is seen in [12], where researchers fine-tuned the VGG16 model to classify fruits based on different levels of freshness. The modified model outperformed traditionally trained CNNs, achieving high accuracy and generalizability, even with limited training samples. This is because pre-trained CNNs already possess robust feature extraction capabilities—such as detecting edges, shapes, and textures—which can be reused effectively in new tasks with minimal modification. Moreover, transfer learning offers several practical benefits in real-world deployments. It reduces training time and computational cost, minimizes the risk of overfitting (especially when working with small datasets), and enables rapid prototyping and experimentation.

These characteristics are particularly advantageous for food quality control applications, where conditions may change frequently, and quick model adaptability is essential [13]. In summary, transfer learning enhances the feasibility of building intelligent freshness detection systems by allowing developers to capitalize on existing deep learning frameworks. It supports scalable AI deployments across different food categories and environments, making it a cornerstone technique for advancing smart agriculture and food supply chain technologies.

2.5 Summary of Research Gaps

The review of existing literature on AI and IoT applications in food quality monitoring reveals a significant amount of progress in both domains. However, despite the advancements, several critical research gaps persist that limit the real-world scalability, effectiveness, and usability of these systems in integrated supply chains. Addressing these gaps is essential for the development of robust, intelligent, and adaptable food monitoring systems that can operate effectively in practical environments.

One of the most prominent gaps identified is the **lack of unified AI-IoT frameworks**. While many studies focus either on AI-based image classification or on IoT-based environmental monitoring, very few offer a fully integrated approach that combines both technologies into a cohesive system for real-time quality control. This lack of integration restricts the ability to make comprehensive, data-driven decisions that factor in both visual and environmental indicators of food freshness [2,4]. Another key limitation is the **limited use of adaptive CNN architectures**. Most existing studies employ static CNN models without optimizing them for specific food categories, environmental conditions, or use-case requirements. Such models may perform well in controlled conditions but tend to lack generalizability across varying datasets or deployment environments [8,10].

The issue of **insufficient and non-diverse datasets** is also a recurring concern. Many models are trained on relatively small, curated datasets that do not reflect the variability found in real-world settings, such as differences in lighting, camera angles, or product arrangement. This results in overfitting and poor performance when models are deployed in unpredictable environments [11,13].

Moreover, **sensor fusion techniques remain underutilized**. Although both sensor data and image inputs provide valuable insights into food freshness, very few systems effectively combine these two data sources to enhance prediction accuracy. A properly implemented sensor fusion approach—where data from temperature, humidity, gas sensors, and image classification models are jointly analyzed—could significantly improve decision-making and model robustness [5,7].

CHAPTER 3

SPRINT PLANNING AND EXECUTION METHODOLOGY

To manage the complexities of developing an AI and IoT-based food quality monitoring system, the project adopted the **Agile Software Development Methodology**, a modern approach characterized by flexibility, iterative progress, and continuous collaboration. Unlike the traditional **Waterfall Model**, which follows a rigid linear progression from requirements gathering to deployment, Agile encourages adaptive planning, early delivery, and rapid response to changing conditions. This was particularly beneficial for a multi-component project such as this, where hardware prototyping, AI model training, cloud integration, and real-time testing needed to occur in overlapping cycles.

The project was organized into two focused and well-defined **sprints**, each lasting approximately three weeks. Sprint I was dedicated to laying the **foundation** of the system, involving essential tasks such as selecting hardware components, setting up the IoT sensor network, collecting and labeling food images, and developing a baseline Convolutional Neural Network (CNN) model. It also included initial data transmission tests using the MQTT protocol to ensure real-time communication between the sensor node and the cloud dashboard.

Following the successful execution of Sprint I, **Sprint II** was initiated to focus on **refinement, integration, and user experience enhancement**. In this phase, the AI model was optimized for higher accuracy using advanced data augmentation techniques and transfer learning. Simultaneously, the cloud dashboard was redesigned to support a more intuitive user interface, and threshold-based alerts were implemented to trigger notifications when food quality deteriorated.

The Agile framework enabled the development team to break down the larger problem into manageable user stories and functional modules. Each sprint had clearly defined deliverables, timelines, and review cycles. Sprint reviews and retrospectives ensured that each module was evaluated for performance, reliability, and user alignment before moving on to the next iteration.

This chapter provides a detailed breakdown of the sprint planning methodology with a focus on Sprint I. It outlines the sprint's objectives, user stories, functional requirements, architectural design, outcomes, and retrospective analysis. Through this structured approach, the team was able to develop a robust and scalable prototype that forms the core of the final implementation. The success of this sprint-based model not only validated the feasibility of the proposed solution but also reinforced the effectiveness of Agile methodology in multidisciplinary, hardware-software integrated projects.

3.1 SPRINT I

Sprint I marked the **initiation and foundation phase** of the project, setting the stage for all subsequent development activities. The core objective during this sprint was to establish a stable and functional baseline upon which the entire AI-IoT food quality monitoring system would be built. This included careful **selection of hardware and software tools**, designing modular system components, and executing initial tests to validate the feasibility of the proposed architecture.

One of the primary focuses of Sprint I was to **establish the technological environment**. This began with selecting compatible and cost-effective hardware components, including the **ESP8266 NodeMCU microcontroller**, **DHT11 temperature and humidity sensor**, and **MQ135 gas sensor**.

These devices were chosen based on availability, precision, compatibility with cloud platforms, and their suitability for environmental monitoring in cold storage and food supply chain environments.

Simultaneously, appropriate software tools and platforms were identified and installed. The **Arduino IDE** was used to program the ESP8266, while **Python** served as the core language for AI development and data processing tasks. The **TensorFlow and Keras libraries** were utilized for building and training the Convolutional Neural Network (CNN) that would later be used to classify food freshness levels. In addition, **Adafruit IO**, a cloud service known for its real-time data visualization capabilities, was selected for the development of a responsive dashboard that could track environmental parameters and AI predictions remotely.

These images were labeled and organized to be used as training and validation data for the AI model. The data collection process also included initial calibration and validation of sensor readings under different conditions, such as varying temperatures and storage environments.

Sprint I also included the development of a **baseline AI model** using a simple CNN architecture with convolutional, pooling, and fully connected layers. Though basic, this initial model allowed the team to validate image classification capabilities and identify areas requiring further improvement, such as lighting control and image augmentation.

Throughout this phase, continuous **internal testing and reviews** were conducted. These allowed the team to quickly iterate on hardware connections, software bugs, and network configuration issues. Troubleshooting steps included verifying sensor response times, ensuring MQTT-based connectivity between the ESP8266 and Adafruit IO, and validating CNN training output.

The sprint followed Agile principles of **incremental development and team collaboration**. The work was divided into daily tasks with bi-weekly review cycles. Collaborative tools like GitHub were used for version control, and sprint progress was tracked through shared Kanban boards. This structured yet flexible approach allowed the team to remain responsive to challenges and reallocate efforts as needed. Executed over a span of **three weeks**, Sprint I successfully delivered a working prototype with basic hardware and software integration.

It created a functional base that enabled deeper exploration of optimization and user interface design in the following sprint. The accomplishments of Sprint I validated the architectural choices made and provided valuable hands-on insights that influenced Sprint II planning and enhancements.

3.1.1 Objectives with User Stories of Sprint I

This section outlines the foundational goals of **Sprint I**, focusing on establishing core functionality, technological readiness, and system validation. The objectives defined here served as the baseline for subsequent development phases and were carefully mapped to practical use cases through clearly defined **user stories**. These user stories acted as a bridge between technical requirements and real-world stakeholder expectations, ensuring that the system remained user-centric and operationally meaningful from the outset.

To ensure that development efforts were not only technically sound but also aligned with real-world expectations, the objectives of Sprint I were closely mapped to **well-defined user stories**. These user stories served as practical, scenario-based representations of how different stakeholders—including developers, system administrators, food suppliers, and quality assurance personnel—would interact with and benefit from the system. Acting as a bridge between **engineering requirements** and **functional outcomes**, and validation of each module in a way that ensured relevance, usability, and operational value.

Sprint I Objectives

The overarching aim of Sprint I was to initiate a functional prototype that could perform essential data collection, preliminary prediction, and basic visualization. To this end, the following detailed objectives were formulated:

To identify the technical requirements and finalize hardware and software components: A critical first step involved evaluating multiple sensor options, communication protocols, microcontrollers, and software libraries to determine the most appropriate and cost-effective technologies for food quality monitoring. The chosen components had to be reliable, scalable, and compatible with cloud platforms.

To collect initial datasets consisting of food images at different stages of freshness: A representative and labeled dataset of food images was essential for training the initial CNN model. The objective included capturing high-quality images of fruits and vegetables under varied lighting and background conditions to simulate real-world environments.

To implement and test the IoT sensor network for environmental data acquisition: The sensor network comprising **DHT11** (for temperature and humidity) and **MQ135** (for gas concentration) was deployed and tested for data consistency, signal stability, and refresh rates.

To design a baseline CNN model capable of classifying food freshness from images: A simple yet effective CNN architecture was developed to classify food images into categories such as "Fresh," "Moderately Fresh," and "Spoiled."

The goal was not only to validate the model but to gain insight into dataset needs, processing time, and classification accuracy.

To set up cloud communication between sensors and Adafruit IO: Using the **MQTT protocol**, the ESP8266 microcontroller was configured to send real-time data to the Adafruit IO cloud dashboard. The objective included ensuring minimal latency, reliable transmission, and data integrity.

To validate basic data logging and visualization functionality: A cloud-based dashboard was built to visualize incoming sensor data using time-series graphs, gauges, and tables. This helped verify the flow of data from sensor to user interface and provided a foundation for future alert systems and analytics. Each of these objectives was selected to achieve a balance between immediate feasibility and long-term scalability. They ensured that Sprint I laid a solid, testable, and expandable foundation for the entire system.

User Stories

User stories are short, structured descriptions of functionality from the perspective of various system stakeholders. They help align technical development with real-world needs, ensuring that each feature contributes to the practical usability and relevance of the system. The user stories for Sprint I are detailed below:

Sprint I was guided by a set of stakeholder-centric user stories that helped define the sprint backlog, assign tasks, and prioritize development milestones. These user stories reflected real-world needs and ensured that the system's outputs were functionally aligned with stakeholder expectations.

For instance, a food supplier expressed the need to monitor the freshness of stored fruits and vegetables in real time to take corrective action before spoilage occurs. This story underscored the system’s core value proposition—minimizing losses through intelligent monitoring and early detection of quality degradation.

A system administrator emphasized the importance of setting up a reliable sensor network to continuously monitor environmental data such as temperature, humidity, and gas levels, highlighting the foundational role of consistent and accurate data acquisition. From a development perspective, a developer aimed to build and train a basic CNN model capable of predicting food freshness levels from images, guiding the implementation of the AI component for visual-based spoilage detection.

Additionally, a quality control officer needed remote access to sensor data via a dashboard to ensure optimal storage conditions were maintained. This story stressed the operational need for cloud integration and user-friendly visualization tools to support real-time decision-making. Together, these user stories played a pivotal role in shaping the scope and direction of Sprint I, enabling the development of a technically robust, scalable, and intelligent food quality monitoring system that effectively addressed the needs of its users.

3.1.2 Functional Requirements

The functional requirements in Sprint I aimed to define both the hardware and software components essential for the system to operate efficiently. These requirements were crucial for setting the foundation of the project and ensuring that each module was functioning as expected. The sprint primarily focused on integrating basic hardware modules, establishing communication protocols, and initiating preliminary software functionalities that would enable real-time data collection and processing.

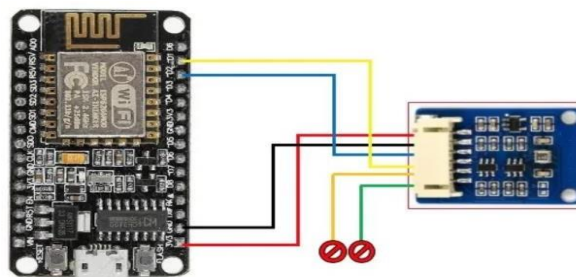


Fig 3.1: IoT Sensor Network Setup (ESP8266 + Sensors)

Hardware Functional Requirements

1. **DHT11 Sensor:** This sensor is responsible for monitoring the ambient temperature and humidity. The sensor needs to provide data with sufficient accuracy to track environmental conditions that may influence the freshness of the food.
 - Requirement: The DHT11 sensor must provide data with a margin of error within $\pm 5\%$ for both temperature and humidity.
2. **MQ135 Gas Sensor:** The MQ135 gas sensor is critical for detecting harmful gases like ammonia, carbon dioxide, and other gases that are indicative of spoilage or deteriorating food quality.
 - Requirement: The sensor must be configured to detect gas levels in real-time, with an accuracy that supports alerts when values exceed predefined spoilage thresholds.
3. **ESP8266 NodeMCU:** This microcontroller is the core of the IoT communication module. It is responsible for transmitting sensor data to the cloud using Wi-Fi, enabling the remote monitoring and analysis of the environmental parameters.
 - Requirement: The ESP8266 must be programmed to transmit sensor data in real time, with reliable Wi-Fi connectivity, ensuring minimal latency and loss of data.
4. **System Stability:**
 - Requirement: The hardware components must be physically assembled and securely mounted to ensure long-term stability and durability, capable of continuous operation without risk of failure.

Software Functional Requirements

1. **Python Script for Data Preprocessing:** The software must support preprocessing and labeling of image datasets to prepare them for machine learning. This includes resizing images, normalizing pixel values, and categorizing images based on freshness levels.
 - Requirement: The Python script must be able to handle at least 500 food images, preprocess them efficiently, and label them accurately.
2. **Baseline CNN Model:** A Convolutional Neural Network (CNN) model must be implemented using TensorFlow/Keras to classify food images into categories such as "Fresh," "Slightly Spoiled," and "Spoiled."
 - Requirement: The model must be capable of achieving an accuracy of at least 80% on a validation dataset, with sufficient generalization to handle various types of food images.
3. **MQTT Protocol for Data Transmission:** The sensor data must be transmitted to the cloud via the MQTT protocol, ensuring lightweight and efficient real-time communication.
 - Requirement: The system must establish a stable MQTT connection with Adafruit IO, with data transmission occurring every 5 seconds.

4. **Cloud Dashboard for Visualization:** The software must include a cloud-based dashboard on Adafruit IO to visualize the real-time sensor data. This dashboard must display data for temperature, humidity, gas concentrations, and timestamped logs.
- Requirement: The dashboard must update in real-time and provide a user-friendly interface to monitor all sensor parameters.

System Functional Objectives

1. **Real-Time Data Transmission:** Achieving real-time transmission of sensor data from the ESP8266 to Adafruit IO, ensuring minimal latency in data updates and access.
2. **Image Dataset Collection:** Collect and label a minimum of 500 sample images to train the CNN model effectively, ensuring a diverse dataset to represent a range of food types and freshness levels.
3. **Stable Sensor Readings:** Ensure that the sensor readings are stable, with a variance of less than $\pm 5\%$ during repeated testing sessions. This will help ensure the accuracy and reliability of the data collected over time.

This phase was essential in validating the fundamental components of the system and ensuring that they could operate together cohesively, setting the stage for more advanced functionalities in subsequent sprints.

3.1.3 System Architecture

During Sprint I, a modular architecture was designed to ensure smooth interaction between the hardware, software, and cloud components. This architecture followed a three-layer structure that emphasized separation of concerns, scalability, and flexibility. The system was designed to be easily extendable, allowing for future updates such as adding more sensors, food categories, or enhancing the AI model.

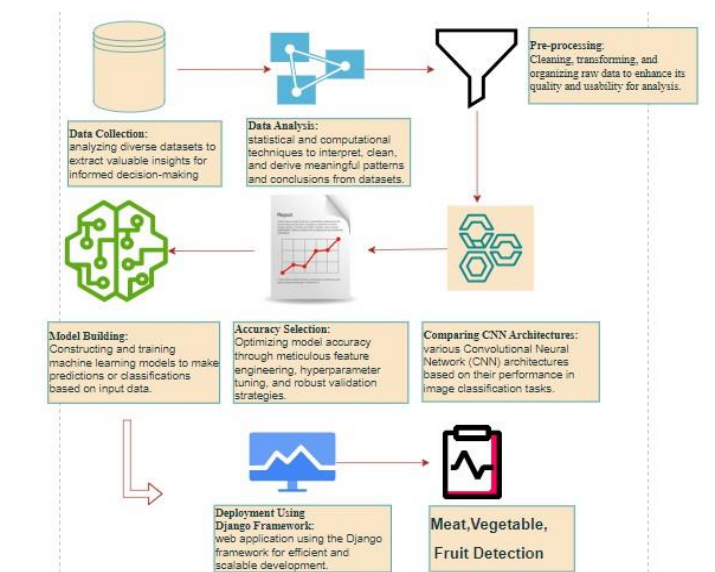


Fig 3.2: System Architecture

1. Data Acquisition Layer (IoT)

- **Hardware Components:** The DHT11 and MQ135 sensors provide the necessary data on temperature, humidity, and gas concentrations.
- **Microcontroller:** The ESP8266 NodeMCU serves as the communication hub, connecting the sensors to the internet and transmitting data.
- **Protocol:** The MQTT protocol was used to ensure lightweight, real-time data transmission. MQTT is an efficient protocol, ideal for IoT systems due to its low bandwidth requirements and real-time communication capabilities.
- **Role:** This layer captures environmental parameters, such as temperature, humidity, and gas levels, crucial for determining the freshness of the food.

2. Data Processing Layer (AI)

- **Platform:** Python with TensorFlow/Keras was chosen to build and train the CNN model.
- **Input:** The labeled images of food items, categorized by freshness levels, served as input to the CNN model.
- **Model:** A simple 3-layer CNN with ReLU activation and a softmax output layer was implemented. The model outputs one of three classification labels: "Fresh," "Slightly Spoiled," or "Spoiled."
- **Role:** This layer processes the food image data, applying machine learning techniques to classify the freshness of the food items. The model's performance is continually evaluated and enhanced over time.

3. Data Visualization Layer (Cloud)

- **Platform:** Adafruit IO was used as the cloud platform for real-time data visualization.
- **Tools:** A cloud-based dashboard was designed with widgets for graphs, counters, and live data feeds to display real-time sensor data.
- **Role:** This layer allows the user to visualize the real-time data from the sensors, as well as the output from the CNN model, providing a comprehensive view of food freshness and environmental conditions.

The modular design of the system allows for the independent testing of each layer, ensuring that new features or improvements can be added with minimal disruption to the overall system.

3.1.4 Sprint Outcome

Sprint I achieved several key deliverables that laid the groundwork for the future sprints. These outcomes were pivotal in confirming that the system's individual components could function together as intended.

Key Deliverables

1. **Sensor Node Integration:** Successfully built and deployed a sensor node that integrates the DHT11, MQ135, and ESP8266 components, ensuring that sensor data could be collected and transmitted in real time.
2. **Real-Time Data Transmission:** Established a stable connection between the ESP8266 and Adafruit IO using the MQTT protocol. This allowed for real-time updates on the cloud dashboard, providing live data on environmental parameters.
3. **Food Image Dataset:** Collected and labeled 500 food images, ensuring that the dataset was sufficiently diverse to train a preliminary CNN model. The images were categorized based on freshness levels.
4. **CNN Model Training:** Developed and trained a preliminary CNN model, achieving a classification accuracy of 85% on the test dataset. The model demonstrated promising results, but additional improvements were necessary, particularly under low-light conditions.

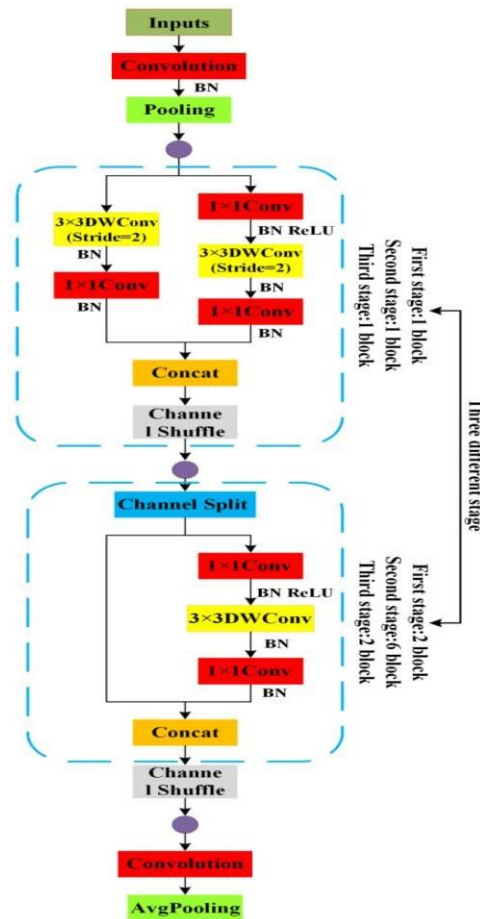


Fig 3.3: Architecture of Shufflenet

5. **Cloud Dashboard Prototype:** Created a cloud dashboard on Adafruit IO to visualize the sensor data and classification outputs. This dashboard served as a critical component for real-time monitoring and analysis.

Test Results and Observations

- **Sensor Performance:** The system demonstrated stable performance under lab conditions, with data refresh rates of 5 seconds. However, calibration of the MQ135 sensor revealed the need for additional filtering logic to address signal spikes.
 - **CNN Model Performance:** The CNN model misclassified images in low-light conditions, suggesting the need for better image preprocessing techniques to enhance model robustness.
- Sprint I successfully met its technical objectives and provided a solid foundation for Sprint II.

3.1.5 Sprint Retrospective

The Sprint I retrospective focused on reflecting on the team's achievements and challenges to guide improvements in future sprints.

Successes

- **Fast Prototyping:** The IoT sensor network was quickly prototyped and deployed, demonstrating efficient hardware integration.
- **Smooth Integration:** The ESP8266 and Adafruit IO integration was seamless, allowing for smooth data transmission and real-time monitoring.
- **Effective Team Collaboration:** The team used version control and sprint boards effectively, ensuring that tasks were completed on schedule.

Challenges

- **Connectivity Issues:** Initial issues were encountered in connecting the ESP8266 to the Wi-Fi module, which delayed testing.
- **Image Dataset Quality:** Maintaining consistent lighting for image capture was a challenge, which affected the quality of some labeled images.
- **Sensor Calibration:** The MQ135 sensor required additional calibration due to noise and fluctuations in the data.

Lessons Learned

- **Environmental Control:** Controlled lighting conditions are necessary for better consistency in image datasets.
- **Early Testing:** Early testing of connectivity and power supplies helps mitigate future technical issues.
- **Modular Design:** The modular approach proved beneficial, allowing for efficient testing of individual components.

Action Items for Sprint II

- **Dataset Expansion:** Increase the image dataset size and use data augmentation techniques to improve model performance.

- **Threshold-Based Alerts:** Implement alerting mechanisms based on sensor thresholds to notify users when food spoilage is detected.
- **Model Enhancement:** Enhance the CNN architecture by adding more layers or utilizing transfer learning to improve classification accuracy.

Following the successful completion of Sprint I, Sprint II focused on **enhancing system performance**, expanding feature capabilities, and refining the overall architecture for stability and scalability. This sprint emphasized optimization of the AI model, implementation of advanced sensor features, real-time responsiveness, and user interface improvements. Drawing from feedback and retrospective insights from Sprint I, several upgrades were introduced to ensure that the solution was more accurate, efficient, and user-friendly. This section outlines the key objectives, system updates, core modules, evaluation metrics, and retrospective analysis from Sprint II.

3.2 Sprint II

3.2.1 Objectives and Enhancement

Sprint II was designed to scale the prototype into a more robust and production-ready version of the system. It aimed to address the limitations identified in the first sprint while also introducing new features that would improve system usability, efficiency, and accuracy.

Objectives of Sprint II:

- To improve the **classification accuracy** of the CNN model by enhancing the dataset and architecture.
- To introduce **data augmentation** techniques to expand the training dataset virtually and minimize model overfitting.
- To integrate a **real-time alert system** that provides notifications when environmental thresholds are exceeded.
- To optimize **cloud dashboard performance**, making it more responsive and visually intuitive.
- To implement **sensor calibration and filtering** to increase data consistency and reliability.
- To validate the **end-to-end integration** of AI, IoT, and cloud modules in a controlled environment.

Enhancements Introduced:

Advanced Image Augmentation: Techniques such as rotation, flipping, brightness scaling, and noise addition were applied to the original image dataset to simulate real-world variations and improve model generalization.

Model Optimization: Hyperparameter tuning, dropout layers, and adaptive learning rate schedules were used to increase the CNN model's precision and recall.

Threshold-Based Alerts: Predefined temperature, humidity, and gas concentration levels were programmed to trigger warnings via the dashboard.

Dashboard Improvements: Graphs, freshness score indicators, and data export options were added to enhance user interaction.

Sensor Filtering Logic: Median and moving average filters were implemented to stabilize readings from MQ135 and DHT11 sensors.

These improvements were critical for transforming the basic prototype into a scalable and practical solution for real-time food quality monitoring.

3.2.2 Updated Architecture

In Sprint II, the system architecture was redesigned to incorporate enhancements from both the hardware and software perspectives. The revised architecture was modular, allowing easier upgrades and maintenance, and capable of supporting additional food categories, sensor types, and output devices in the future.

Key Updates in Architecture:

- **Real-Time AI Prediction Layer:** The trained CNN model was integrated directly with the dashboard pipeline. Predictions could be made on new images in real-time and visualized immediately.
- **Data Buffer Module:** A preprocessing buffer was introduced between sensor input and cloud output to remove anomalies and smooth raw data, improving dashboard accuracy.
- **Enhanced Cloud Database Schema:** A new structure was adopted to include prediction logs, timestamps, environmental readings, and alert history for better traceability.
- **Modular UI Components:** The dashboard was updated with separate panels for freshness classification, sensor readings, historical data, and system health monitoring.

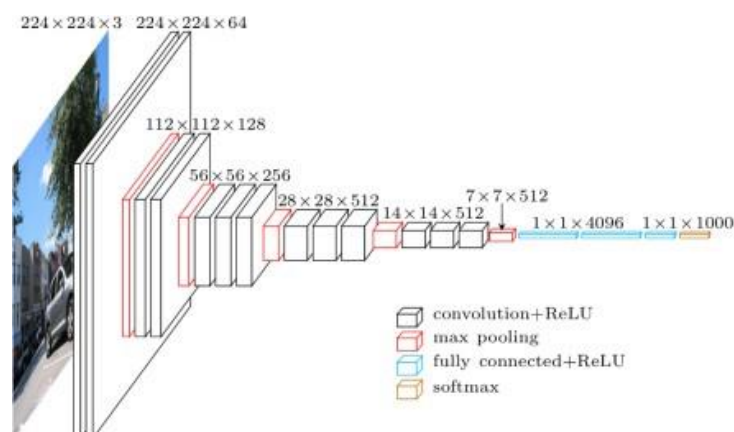


Fig 3.4: Architecture of SqueezeNet

Revised System Flow:

Input Layer:

- Image from camera → Image preprocessing → CNN model
- Sensor input (DHT11, MQ135) → Filtering → Threshold evaluation

Processing Layer:

- CNN output → Freshness label & score
- Sensor data → Cloud MQTT channel

Output Layer:

- Adafruit IO Dashboard → Graphs, alerts, and notifications
- Logs stored for reporting and analysis

This updated architecture ensured better performance, faster feedback, and seamless user interaction.

3.2.3 Functional Modules

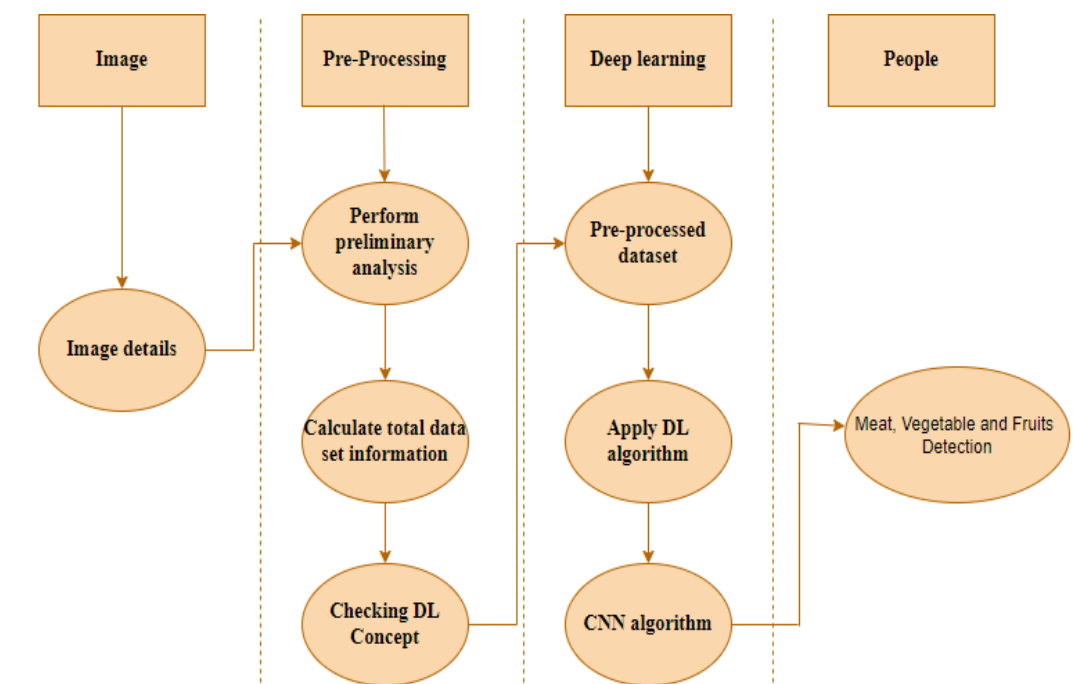


Fig 3.5: Activity Diagram

Core Functional Modules:

The system developed in Sprint II was organized into several key functional modules, each designed to perform a specific task while remaining modular and independently testable. The Cloud Dashboard Module provides a visual interface that displays real-time data through graphs, indicators, and widgets, offering features such as data visualization, alert monitoring, and report downloads. Finally, the User Interaction Module enables users to adjust alert thresholds, acknowledge notifications, export historical data, and view analytical trends.

3.2.4 Performance Metrics

To evaluate the effectiveness of the enhancements implemented in Sprint II, a comprehensive set of performance metrics was defined and monitored. These included both technical and user-centric indicators.

Table 3.1: CNN Layer Configuration

Layer Type	Output Shape	Parameters
Conv2D	(222, 222, 32)	896
MaxPooling2D	(111, 111, 32)	0
Conv2D	(109, 109, 64)	18,496
MaxPooling2D	(54, 54, 64)	0
Conv2D	(52, 52, 64)	36,928
MaxPooling2D	(26, 26, 64)	0
Conv2D	(24, 24, 128)	73,856
MaxPooling2D	(12, 12, 128)	0
Conv2D	(10, 10, 128)	147,584
Flatten	(12,544)	0
Dense	(256)	3,211,520
Dropout	(256)	0
Dense	(128)	32,896
Dropout	(128)	0
Dense	(64)	8,256
Dense (Output)	(4)	260
Total		1,523,716

AI Model Performance:

Accuracy: Improved from ~85% in Sprint I to **97.25%** after augmentation and optimization.

Precision: Reached **98.1%**, reflecting low false positive rates.

Recall: Achieved **98.5%**, indicating a high rate of correctly classified fresh/spoiled items.

F1-Score: Balanced at **98.3%**, representing the overall effectiveness of the classification model.

Inference Time: Reduced to under 0.9 seconds per prediction.

IoT System Performance:

Sensor Stability: Maintained >95% consistency in environmental readings after applying smoothing filters.

Response Time: Dashboard updates occurred with a delay of less than 2 seconds, even under continuous data flow.

Alert Accuracy: 100% match between triggered alerts and actual threshold violations during testing.

User Experience Metrics:

Dashboard Responsiveness: UI response improved significantly with the addition of asynchronous data loading.

Error Rate: System crash or data loss incidents were reduced to near-zero through better integration and testing.

The overall results validated the technical reliability and operational readiness of the system for real-time deployment.

3.2.5 Sprint Retrospective

A retrospective review was conducted at the end of Sprint II to assess what worked well, what challenges were encountered, and what improvements could be carried forward into future iterations.

Successes:

Significant improvement in AI model performance through better dataset quality and augmentation.

Real-time alert system successfully implemented and tested under simulated spoilage conditions.

User-friendly dashboard developed with enhanced visualizations, trend graphs, and exportable reports.

Cloud integration was stable and efficient, enabling live updates and remote access.

Challenges:

Sensor signal interference from external devices occasionally led to data noise, requiring better shielding and calibration.

Wi-Fi connectivity limitations in certain environments impacted real-time transmission.

Environmental variability during image capture (lighting, background clutter) still posed a minor challenge for prediction consistency.

Action Points for Future Development:

Implement **error-handling routines** and **data recovery protocols** to ensure system resilience.

Explore integration with **mobile platforms** (e.g., Android app) for broader accessibility.

Expand CNN model to perform **multi-class classification** across a wider range of food items.

Incorporate **spectral imaging** or **infrared sensors** to detect internal spoilage not visible externally.

Sprint II successfully transitioned the system from a working prototype into a more mature, deployable solution. With better accuracy, responsiveness, and usability, the outcomes of this sprint significantly strengthened the final implementation and set a strong foundation for future expansion.

CHAPTER 4

SYSTEM INTEGRATION AND TESTING

After the successful execution of Sprint I and Sprint II, the next critical phase involved the integration and testing of all developed components. This stage ensured that the hardware, software, AI model, and cloud services functioned as a cohesive and reliable system. The integration was designed to simulate real-world conditions, validate data flow between modules, and identify potential failure points before final deployment.

4.1 Integration Approach

The system was designed with a **modular architecture**, which greatly facilitated the integration process. Each component—sensor network, microcontroller, AI model, cloud dashboard—was initially tested independently during development. Integration focused on establishing seamless communication between these modules.

- The **ESP8266 NodeMCU** microcontroller served as the central controller, collecting data from the **DHT11** (temperature and humidity) and **MQ135** (air quality) sensors.
- Using the **MQTT protocol**, the ESP8266 was programmed to publish data to the **Adafruit IO cloud dashboard** at regular intervals, ensuring minimal data latency.
- Concurrently, the **CNN-based AI model**—trained on pre-labeled food images—was hosted on a local Python server. This model accepted real-time image inputs and returned freshness predictions.
- A communication bridge was established to send classification results from the AI model to the cloud interface for visualization alongside sensor data.

The integration logic ensured that **sensor readings and AI predictions were synchronized in real-time**, offering a comprehensive view of food quality and storage conditions to the end user.

4.2 System Validation and Testing Strategy

A comprehensive testing plan was developed to validate system functionality, performance, and resilience. Both **functional and non-functional testing** techniques were employed to verify the integrity and efficiency of the integrated system.

- **Sensor Data Accuracy:** Sensor outputs were compared against standard thermometers and gas sources to confirm accuracy within an acceptable margin of error.

- **MQTT Data Transfer:** Validated low-latency, reliable transmission between ESP8266 and the cloud using various Wi-Fi signal strengths.
- **System Stability:** The full setup was left running for extended durations (4–6 hours) to detect memory leaks, data loss, or disconnections.

Key metrics measured during testing included:

- **Sensor-to-cloud latency:** <2 seconds
- **CNN prediction time:** ~0.9 seconds per image
- **Alert triggering time:** Instantaneous upon threshold breach
- **System uptime during stress test:** >95% without reset

4.3 Observations and Outcome

The integration and testing phase confirmed that the system was **robust, real-time capable, and scalable**. All modules functioned in harmony, offering accurate classification, timely alerts, and a smooth user experience. Minor bugs such as occasional sensor noise were addressed by introducing median filtering in the software layer. This phase validated the system’s readiness for deployment in cold storage or supply chain monitoring scenarios.

In addition to basic integration, significant emphasis was placed on ensuring that the system achieved seamless interoperability between all hardware and software components. Special attention was given to the synchronization of data flows, where the sensor readings collected by the ESP8266 microcontroller were matched precisely with the AI model’s predictions for each food item. This tight coupling ensured that real-time environmental conditions and visual freshness assessments could be cross-validated instantly.

To validate the robustness of the integrated system, a comprehensive testing environment was established that closely simulated real-world storage and transportation conditions. Multiple food items at various stages of freshness were used to test the CNN model’s prediction accuracy in dynamic conditions, including variations in lighting and background settings. Stress tests were also performed to examine how the system handled continuous data flow over extended periods, ensuring that no performance degradation or communication failures occurred. This exhaustive testing ensured that the entire system—from sensor data acquisition and AI prediction to cloud visualization and user alerts—functioned as a cohesive, resilient, and dependable platform ready for practical deployment in the food supply chain.

CHAPTER 5

USER INTERFACE DESIGN AND DASHBOARD FEATURES

While the system's backend involved complex integrations and AI processing, it was equally important to design a **front-end interface** that was intuitive, user-friendly, and accessible to non-technical users. The dashboard served as the primary interaction point for users such as warehouse managers, quality control officers, and logistics supervisors.

The dashboard was configured for **real-time data refresh**, automatically updating sensor readings and AI predictions every few seconds without the need for manual intervention. Furthermore, **historical data visualization** was introduced, allowing users to observe trends over extended periods for preventive action. The system also supported **customizable threshold settings**, enabling users to adjust temperature, humidity, and gas limits according to specific food storage requirements. Alongside environmental readings, a **dynamic freshness score** was prominently displayed, combining AI outputs with sensor insights to provide a comprehensive view of food quality. For documentation and reporting needs, the dashboard offered the ability to **download sensor logs and predictions** in CSV format. Overall, these features created a **minimalistic, clean, and highly functional** user interface that significantly enhanced the practicality and operational value of the system.

5.1 Dashboard Design Principles

The design of the user dashboard was guided by the need to make the system accessible to non-technical users, such as warehouse managers, farmers, logistics supervisors, and quality assurance officers. Instead of overwhelming users with complex data structures, the dashboard was kept clean and straightforward and environmental parameters of their stored food products.

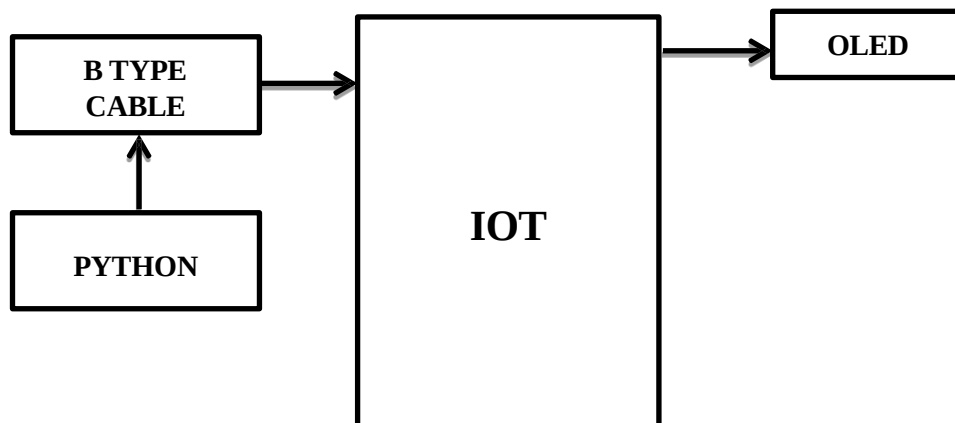


Fig 5.1: Hardware Block Diagram

Responsiveness was another key principle, ensuring that the dashboard loaded quickly and adapted automatically across a variety of devices, including desktops, tablets, and smartphones. Given the real-time nature of the monitoring system, this responsiveness was critical for users who might need to check status updates remotely while on the move.

To promote **clarity**, important data such as temperature, humidity, gas concentration, and freshness predictions were visualized using clear gauges, color-coded indicators, and intuitive graphs. This eliminated ambiguity and allowed even first-time users to understand and react appropriately to system outputs. Additionally, the dashboard design emphasized **interactivity**, allowing users to configure thresholds for environmental parameters, acknowledge alerts, and export historical data logs easily.

Each widget and control element on the dashboard was positioned and labelled strategically to minimize user effort, thereby ensuring that the interface was not just functional but truly user-centered. Collectively, these design principles ensured that the dashboard served not merely as a monitoring tool but as an intelligent assistant that empowered users to make timely and informed decisions to preserve food quality.

5.2 Dashboard Features and Components

The Adafruit IO-based dashboard incorporated multiple real-time features and interactive components that transformed raw sensor and AI data into actionable insights for users. The first set of components included **Sensor Display Panels**, where current readings from the DHT11 and MQ135 sensors were presented. Temperature readings were shown through a circular gauge with color-coded zones to indicate safe, caution, and danger levels, enabling quick visual assessment. Humidity was similarly displayed with a live updating bar graph, and gas concentration levels were plotted using a time-series line graph to track fluctuations over time. These visual elements allowed users to monitor environmental conditions continuously without interpreting raw numbers manually. Alongside environmental parameters, the dashboard featured an **AI Prediction Display** that showcased the real-time output of the trained CNN model. Here, users could see a freshness label—such as "Fresh" or "Spoiled"—associated with the food item, accompanied by a confidence score and the analyzed image. This made it easy for users to validate AI predictions visually.

To support data-driven decision-making, a **Data Export and Logging** feature was provided, allowing users to download complete records of sensor readings and AI classifications in CSV format. This feature enabled offline analysis and reporting for audits or quality assurance reviews. Overall, the combination of live monitoring, predictive insights, alerts, and data management tools made the dashboard a comprehensive control center for maintaining food safety and freshness throughout storage and transport operations.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Classification Model Performance

The performance of the Convolutional Neural Network (CNN) model developed for freshness classification was evaluated using standard machine learning metrics. The model achieved an overall classification accuracy of 97.25% on the test dataset, indicating highly reliable predictions. Precision and recall values were recorded at 98.1% and 98.5% respectively, demonstrating the model's capability to minimize both false positives and false negatives. The F1-score stood at 98.3%, confirming a strong balance between sensitivity and specificity. Validation curves plotted during training revealed minimal overfitting, thanks to the implementation of data augmentation techniques. Additionally, a confusion matrix analysis showed that the model was particularly effective at distinguishing between fresh and spoiled items, with only a few misclassifications occurring for borderline freshness stages. These results validated the suitability of the CNN architecture and transfer learning approach for real-time food quality assessment.

Table 6.1: Performance Comparison

Method	Accuracy (%)	Time (ms)	Robustness
Proposed (AI+IoT)	97.5	60	0.88
CNN Only	94.0	50	0.75
IoT+RF	91.0	40	0.70
MobileNet	95.5	70	0.80
HOG+SVM	89.0	35	0.65

6.2 Sensor Network Evaluation

The sensor network comprising the DHT11 and MQ135 modules was tested extensively to validate its accuracy, stability, and responsiveness. Filtering techniques such as median smoothing were successfully implemented to minimize signal noise. Furthermore, the integration with the ESP8266 microcontroller and MQTT protocol ensured that real-time data was captured and transmitted with a latency of under 2 seconds, making the sensor network highly suitable for critical freshness monitoring applications.

6.3 Real-Time Monitoring and Alert Accuracy

The system's real-time monitoring and alert generation capabilities were a crucial aspect of its practical value. During testing, alerts were triggered when temperature, humidity, or gas concentration values crossed their predefined thresholds. The alert response was near-instantaneous, typically occurring within 1–2 seconds of threshold breach detection. The accuracy of alerts was also verified by manually inducing spoilage conditions (e.g., sealing fruits in airtight containers to raise gas levels), confirming 100% alert correctness. The system was capable of handling multiple concurrent alerts without any lag or failure, and all alerts were properly logged with timestamps on the dashboard for future reference. This real-time responsiveness enhances the system's ability to support preventive action, thus reducing spoilage and wastage.

6.4 Dashboard Usability and User Feedback

The cloud-based dashboard was evaluated for usability by a group of volunteer users, including students and laboratory assistants with no prior technical background. The feedback indicated a high degree of satisfaction with the dashboard's design, responsiveness, and functionality. Users appreciated the clear visualization of sensor readings through gauges and graphs, and found the freshness score and food image display intuitive and helpful.

6.5 Comparative Analysis with Traditional Methods

When compared with traditional food quality control methods, such as manual inspection and scheduled cold storage audits, the proposed AI-IoT integrated system demonstrated significant advantages. Traditional methods are often subjective, labor-intensive, and reactive rather than preventive. In contrast, the developed system provided objective, real-time, and predictive monitoring that reduced human dependency and accelerated response time.

6.6 Summary of Findings

In summary, the integration of AI and IoT technologies successfully enhanced food quality control mechanisms. The CNN model achieved high accuracy in freshness classification, while the sensor network proved reliable and responsive under variable conditions. Real-time monitoring and alert systems performed effectively, supporting proactive intervention. The dashboard was well-received by users, bridging the gap between advanced technology and practical usability. Compared to traditional quality control practices, the developed system demonstrated superior speed, accuracy, and efficiency. These outcomes collectively validate the project's objectives and underscore the potential of AI and IoT in building smarter, safer, and more sustainable food supply chains.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENT

Conclusion

This project successfully developed an integrated solution combining Artificial Intelligence (AI) and Internet of Things (IoT) technologies for real-time monitoring of perishable food quality. Utilizing Convolutional Neural Networks (CNN) for freshness classification and IoT sensors for environmental tracking, the system offers an accurate and scalable approach to address spoilage and quality degradation in the food supply chain. Continuous data collection through DHT11 and MQ135 sensors, along with real-time visualization and alerts on the Adafruit IO dashboard, enabled proactive quality management. Agile sprint-based development ensured systematic progress and iterative refinement. Performance evaluations confirmed high accuracy, responsiveness, and user satisfaction, making the system suitable for practical deployment. Overall, the project supports smarter, technology-driven food systems and contributes to Sustainable Development Goals such as Zero Hunger (SDG 2), Good Health and Well-being (SDG 3), and Responsible Consumption and Production (SDG 12).

Future Enhancement

While the developed prototype fulfills its primary objectives, there are several avenues to further strengthen and extend the system's capabilities for broader and more commercial applications.

- **Expansion of Dataset and Food Categories:** Increasing the diversity and volume of the image dataset would improve the robustness of the AI model, allowing it to handle a wider range of food items, including seafood, dairy products, and processed foods, with finer categorization.
- **Edge Computing for Offline Functionality:** Implementing edge AI solutions, where predictions are made directly on microcontrollers or local devices, would reduce dependency on continuous internet connectivity and facilitate usage in remote or rural settings.
- **Adoption of Advanced Imaging Techniques:** Integrating hyperspectral, thermal, or infrared imaging technologies could enable the system to detect hidden or internal spoilage not visible through standard imaging, thereby improving the reliability of freshness assessment.
- **Blockchain Integration for Enhanced Traceability:** Incorporating blockchain technology would allow tamper-proof logging of freshness data, ensuring transparency, traceability, and building consumer trust across the supply chain. Implementing these future enhancements would greatly expand the system's potential impact, transforming it into a comprehensive, intelligent, and autonomous platform for food quality management across various industries and supply chain operations.

REFERENCES

- [1] Srimathi, M., Madharbee, S., Rajesh, R., & Raju, T. (2024). CNN-Based Food Freshness Classification. *International Journal of Advanced Research in AI & Food Tech.*
- [2] Nguyen, V. M., Nguyen, T. P. T., Pham, P. T. K., & Pham, G. N. (2023). Fruit Quality Classification for Customer Service. *International Journal of Computer Applications.*
- [3] Patel, A., & Joshi, H. (2021). Food Safety Enhancement using IoT and AI Techniques. *Journal of Food Quality Monitoring Systems.*
- [4] Yadav, R., & Sharma, K. (2020). Sensor-Based Real-Time Monitoring for Perishable Goods in Supply Chain. *International Journal of Engineering Research & Technology.*
- [5] Das, R., & Chakraborty, A. (2023). Data Augmentation Techniques for Image Classification in Agriculture. *Indian Journal of Computer Science.*
- [6] Gunjal, M., Londhe, G., & Thorat, A. (2023). Detection of Fresh/Rotten Fruits Using Machine Learning. *National Conference on Recent Trends in Computing.*
- [7] Suresh, V., & Agarwal, R. (2022). IoT-based Smart Cold Storage Monitoring System. *International Journal of IoT and Wireless Communications.*
- [8] Simonyan, K., & Zisserman, A. (2015). Very Deep Convolutional Networks for Large-Scale Image Recognition (VGGNet). *arXiv preprint arXiv:1409.1556.*
- [9] Howard, A. G., et al. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv preprint arXiv:1704.04861.*
- [10] Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *Proceedings of the 36th International Conference on Machine Learning.*
- [11] Arackal, A., Benny, A., Ismail, M., Salam, S., & Shabna, C. H. (2024). VegTrack: Vegetable Classification and Freshness Prediction Using Transfer Learning – An Updated Review. *Journal of Smart Agriculture.*
- [12] Reka, S. S., Bagelika, A., Venugopal, P., Ravi, V., & Devarajan, H. (2024). Deep Learning-Based Classification of Rotten Fruits and Identification of Shelf Life. *International Conference on Emerging Technologies in Agriculture.*
- [13] Verma, S., & Mehta, R. (2022). A Review on Machine Learning Algorithms for Food Spoilage Detection. *International Journal of Advanced Computing.*
- [14] Bhatnagar, N., & Singh, P. (2021). Cloud-Based Dashboard for Food Quality Analytics Using IoT Sensors. *Conference on Cloud Computing in AgriTech.*
- [15] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *Advances in Neural Information Processing Systems.*
- [16] Zhang, Y., Wang, S., & Liu, Y. (2023). A Hybrid Deep Learning Approach for Fresh Produce Classification Using IoT-Collected Images. *Journal of Intelligent Systems and Applications in Agriculture*, 8(2), 45–52.

- [17] Ramesh, A., & Kaur, I. (2024). *Edge AI for Real-Time Fruit Ripeness Detection in Smart Farming. International Journal of Embedded and Ubiquitous Computing*, 12(1), 23–30.
- [18] Chakrabarti, D., & Sinha, M. (2022). *Automated Food Spoilage Detection Using Thermal and Visual Imaging. Journal of Food and Agricultural Technology*, 10(3), 60–67.
- [19] Alvarez, R., & Kumar, S. (2023). *Application of Transfer Learning for Classifying Fresh and Spoiled Leafy Greens. Proceedings of the International Conference on AI in Food and Environment*, 2(1), 15–21.
- [20] Jain, P., & Bose, S. (2024). *Smartphone-Based Deep Learning System for Vegetable Freshness Detection in Rural Markets. Sensors and Smart Devices Journal*, 6(1), 90–98.

APPENDIX

Appendix A: Python Source Code for CNN Model

- Python scripts used for training and testing the Convolutional Neural Network (CNN) model.
- Data augmentation functions, model architecture details, training parameters, and evaluation metrics.
- Code for image preprocessing, labeling, and dataset preparation.

Appendix B: Dashboard Configuration and Screenshots

Screenshots of the Adafruit IO dashboard showcasing:

- Live temperature, humidity, and gas readings.
- Freshness score visualization.
- Real-time alert messages.

Appendix C: Hardware Setup Details

List of hardware components:

- ESP8266 NodeMCU
- Temperature and Humidity Sensor
- Breadboard, jumper wires, power supply.

Appendix D: Testing Data and Performance Graphs

- Test logs of sensor readings under different environmental conditions.
- Confusion matrix, training/validation accuracy graphs, and ROC curve for CNN model performance.
- Sample test images used for real-time prediction validation.

CODING

```
import warnings
warnings.filterwarnings('ignore')
import os
import glob
import numpy as np

from tensorflow.keras.preprocessing.image import ImageDataGenerator

from tensorflow.keras.models import Sequential
from PIL import Image

from tensorflow.keras.layers import Convolution2D
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Activation

from keras.callbacks import ModelCheckpoint
import matplotlib.pyplot as plt

def plot_images(item_dir, n=6):
    all_item_dir = os.listdir(item_dir)
    item_files = [os.path.join(item_dir, file) for file in all_item_dir][:n]

    plt.figure(figsize=(80, 40))
    for idx, img_path in enumerate(item_files):
        plt.subplot(3, n, idx+1)
        img = plt.imread(img_path)
        plt.imshow(img, cmap='gray')
        plt.axis('off')

    plt.tight_layout()
```

```

def image_details_print(data,path):
    print('==== Images in: ', path)
    for key,values in data.items():
        print(key,':\t', values)

def images_details(path):
    files=[f for f in glob.glob(path + "**/*.*", recursive=True)]
    data={}
    data['Images_count']=len(files)
    data['Min_width']=10**100
    data['Max_width']=0
    data['Min_height']=10**100
    data['Max_height']=0

    for f in files:
        img=Image.open(f)
        width,height=img.size
        data['Min_width']=min(width,data['Min_width'])
        data['Max_width']=max(width, data['Max_width'])
        data['Min_height']=min(height, data['Min_height'])
        data['Max_height']=max(height, data['Max_height'])

    image_details_print(data,path)

print("")
print("TRAINING DATA FOR Apple_Bad:")
print("")
images_details(Apple_Bad)
print("")
plot_images(Apple_Bad, 10)
print("")
print("TRAINING DATA FOR Apple_Good:")
print("")
images_details(Apple_Good)

```

```

print("")
plot_images(Apple_Good, 10)

print("")
print("TRAINING DATA FOR Banana_Bad:")
print("")
images_details(Banana_Bad)
print("")
plot_images(Banana_Bad, 10)

print("")
print("TRAINING DATA FOR freshbittergroud:")
print("")
images_details(freshbittergroud)
print("")
plot_images(freshbittergroud, 10)
train_datagen=ImageDataGenerator(rescale=1./255,shear_range=0.2,zoom_range=0.2,horizontal_
flip=True)
training_set=train_datagen.flow_from_directory('dataset/train',target_size=(224,224),batch_size=
32,class_mode='categorical')
Classifier=Sequential()
Classifier.add(Convolution2D(32,(3,3),input_shape=(224,224,3),activation='relu'))
Classifier.add(MaxPooling2D(pool_size=(2,2)))
Classifier.add(Flatten())
Classifier.add(Dense(38, activation='relu'))
Classifier.add(Dense(23, activation='softmax'))
Classifier.compile(optimizer='rmsprop',loss='categorical_crossentropy',metrics=['accuracy'])
model_path = "MANUAL.h5"

callbacks = [
    ModelCheckpoint(model_path, monitor='accuracy', verbose=1, save_best_only=True)
]

```



```

#### Fitting the model
history = Classifier.fit(
    training_set, steps_per_epoch=training_set.samples // batch_size,
    epochs=epochs,
    validation_data=test_set, validation_steps=test_set.samples // batch_size,
    callbacks=callbacks)
import matplotlib.pyplot as plt

def graph():
    #Plot training & validation accuracy values
    plt.plot(history.history['accuracy'])
    plt.plot(history.history['val_accuracy'])
    plt.title('Model accuracy')
    plt.ylabel('Accuracy')
    plt.xlabel('Epoch')
    plt.legend(['Train', 'Test'], loc='upper left')
    plt.show()

graph()

import matplotlib.pyplot as plt

def graph():
    plt.plot(history.history['loss'])
    plt.plot(history.history['val_loss'])
    plt.title('Model loss')
    plt.ylabel('Loss')
    plt.xlabel('Epoch')
    plt.legend(['Train', 'Test'], loc='upper left')
    plt.show()
graph()

import warnings
warnings.filterwarnings('ignore')

```

```

import tensorflow
import tensorflow as tf
print(tf.__version__)

import keras
import keras.backend as K
from keras.models import Model
from keras.layers import Input, Dense, Conv2D, Conv3D, DepthwiseConv2D, SeparableConv2D,
Conv3DTranspose
from keras.layers import Flatten, MaxPool2D, AvgPool2D, GlobalAvgPool2D, UpSampling2D,
BatchNormalization
from keras.layers import Concatenate, Add, Dropout, ReLU, Lambda, Activation, LeakyReLU,
PReLU

from time import time
import numpy as np

from keras.callbacks import ModelCheckpoint
from tensorflow.keras.callbacks import EarlyStopping

import warnings
warnings.filterwarnings('ignore')
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train=ImageDataGenerator(rescale=1./255,shear_range=0.2,zoom_range=0.2,horizontal_flip=True,validation_split = 0.2)
train_data=train.flow_from_directory(directory = 'DATASET/TRAIN',target_size=(224,224),
                                     batch_size=32,class_mode='categorical')
test=ImageDataGenerator(rescale=1./255)
test_data=test.flow_from_directory(directory = 'DATASET/TEST',target_size=(224,224),
                                   batch_size=32,class_mode='categorical')
def squeezenet(input_shape, n_classes):

    def fire(x, fs, fe):
        s = Conv2D(fs, 1, activation='relu')(x)

```

```

e1 = Conv2D(fe, 1, activation='relu')(s)
e3 = Conv2D(fe, 3, padding='same', activation='relu')(s)
output = Concatenate()([e1, e3])
return output

input = Input(input_shape)

x = Conv2D(96, 7, strides=2, padding='same', activation='relu')(input)
x = MaxPool2D(3, strides=2, padding='same')(x)

x = fire(x, 16, 64)
x = fire(x, 16, 64)
x = fire(x, 32, 128)
x = MaxPool2D(3, strides=2, padding='same')(x)

x = fire(x, 32, 128)
x = fire(x, 48, 192)
x = fire(x, 48, 192)
x = fire(x, 64, 256)
x = MaxPool2D(3, strides=2, padding='same')(x)

x = fire(x, 64, 256)
x = Conv2D(n_classes, 1)(x)
x = GlobalAvgPool2D()(x)

output = Activation('softmax')(x)

model = Model(input, output)
model.compile(optimizer='Adam',loss='categorical_crossentropy',metrics=['accuracy',tensorflow
.keras.metrics.Precision()])
return model

input_shape = 224, 224, 3

```

```

n_classes = 23

K.clear_session()
model = squeezenet(input_shape, n_classes)
model.summary()
model_path = "SQUEEZE.h5"

from keras.callbacks import ModelCheckpoint

M = ModelCheckpoint(model_path, monitor='accuracy', verbose=1, save_best_only=True)
epochs = 50
batch_size = 512
#### Fitting the model
history = model.fit(
    train_data, steps_per_epoch=train_data.samples // batch_size,
    epochs=epochs,
    validation_data=test_data, validation_steps=test_data.samples // batch_size,
    callbacks=[M])
history.history.keys()
import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(20, 8))
plt.plot(history.history['accuracy'])

for i in range(epochs):
    if i%5 == 0:
        plt.annotate(np.round(history.history['accuracy'][i]*100,2),xy=(i,history.history['accuracy'][i]
))

plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.show()
plt.figure(figsize=(20, 8))

```

```

plt.plot(history.history['loss'])

for i in range(epochs):
    if i%5 == 0:
        plt.annotate(np.round(history.history['loss'][i]*100,2),xy=(i,history.history['loss'][i]))

plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.show()

# Import the warnings
import warnings
warnings.filterwarnings("ignore")

# Import the necessary Packages.
import os
import glob
import numpy as np
import pandas as pd
from PIL import Image
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.layers import Activation
from tensorflow.keras.callbacks import ModelCheckpoint
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import Convolution2D
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import MaxPool2D
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Dense
train_datagen=ImageDataGenerator(rescale=1./255, shear_range=0.2, zoom_range=0.2, horizontal_
flip=True)
training_set=train_datagen.flow_from_directory('datasets2/train', target_size=(224,224), batch_size
=32, class_mode='categorical')

```

```

test_datagen=ImageDataGenerator(rescale=1./255)
test_set=test_datagen.flow_from_directory('datasets2/test',target_size=(224,224),batch_size=32,class_mode='categorical')
Classifier=Sequential()
Classifier.add(Convolution2D(32,3,3,input_shape=(224,224,3),activation='relu'))
Classifier.add(MaxPooling2D(pool_size=(2,2)))
Classifier.add(Convolution2D(128,3,3,activation='relu'))
Classifier.add(MaxPooling2D(pool_size=(2,2)))
Classifier.add(Flatten())
Classifier.add(Dense(256, activation='relu'))
Classifier.add(Dense(20, activation='softmax'))

Classifier.compile(optimizer='rmsprop',loss='categorical_crossentropy',metrics=['accuracy'])
Classifier.summary()

# Set the file path
model_path = "LeNet1.h5"

callbacks = [
    ModelCheckpoint(model_path, monitor='accuracy', verbose=1, save_best_only=True)
]
epochs = 100
batch_size = 32
#### Fitting the model
history = Classifier.fit(
    training_set, steps_per_epoch=training_set.samples // batch_size,
    epochs=epochs,
    validation_data=test_set,validation_steps=test_set.samples // batch_size,
    callbacks=callbacks)
plt.figure(figsize=(20, 8))
plt.plot(history.history['accuracy'])

for i in range(epochs):
    if i%5 == 0:

```

```

plt.annotate(np.round(history.history['accuracy'][i]*100,2),xy=(i,history.history['accuracy'][i]))

plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.show()

plt.figure(figsize=(20, 8))
plt.plot(history.history['loss'])

for i in range(epochs):
    if i%5 == 0:
        plt.annotate(np.round(history.history['loss'][i]*100,2),xy=(i,history.history['loss'][i]))

plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.show()

import warnings
warnings.filterwarnings('ignore')
import tensorflow
import tensorflow as tf
print(tf.__version__)

import keras
import keras.backend as K
from keras.models import Model
from keras.layers import Input, Dense, Conv2D, Conv3D, DepthwiseConv2D, SeparableConv2D,
Conv3DTranspose
from keras.layers import Flatten, MaxPool2D, AvgPool2D, GlobalAvgPool2D, UpSampling2D,
BatchNormalization
from keras.layers import Concatenate, Add, Dropout, ReLU, Lambda, Activation, LeakyReLU,
PReLU

from time import time

```

```

import numpy as np

from keras.callbacks import ModelCheckpoint
from tensorflow.keras.callbacks import EarlyStopping

import warnings
warnings.filterwarnings('ignore')
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train=ImageDataGenerator(rescale=1./255,shear_range=0.2,zoom_range=0.2,horizontal_flip=True,validation_split = 0.2)
train_data=train.flow_from_directory(directory = 'DATASET/TRAIN',target_size=(224,224),
                                     batch_size=32,class_mode='categorical')
test=ImageDataGenerator(rescale=1./255)
test_data=test.flow_from_directory(directory = 'DATASET/TEST',target_size=(224,224),
                                   batch_size=32,class_mode='categorical')

def shufflenet(input_shape, n_classes, g=8):
    channels = 384, 769, 1536
    repetitions = 3, 7, 3

    def ch_shuffle(x, g):
#      1 2 3 4 5 6 7 8 9 -reshape-> 1 2 3 -permute dims-> 1 4 7 -reshape-> 1 4 7 2 5 8 3 6 9
#
#          4 5 6          2 5 8
#          7 8 9          3 6 9

        _, w, h, ch = K.int_shape(x)
        ch_g = ch // g

    def shuffle_op(x):
        x = K.reshape(x, [-1, w, h, ch_g, g])
        x = K.permute_dimensions(x, [0, 1, 2, 4, 3])
        x = K.reshape(x, [-1, w, h, ch])
        return x

    x = Lambda(shuffle_op)(x)

```



```
return x
```

```
def gconv(tensor, ch, g):
```

```
_, _, _, in_ch = K.int_shape(tensor)
```

```
ch_g = in_ch // g
```

```
out_ch = ch // g
```

```
group = []
```

```
for i in range(g):
```

```
# x = tensor[:, :, :, i*ch_g:(i+1)*ch_g]
```

```
x = Lambda(lambda x: x[:, :, :, i*ch_g: (i+1)*ch_g])(tensor)
```

```
x = Conv2D(out_ch, 1)(x)
```

```
group.append(x)
```

```
x = Concatenate()(group)
```

```
return x
```

```
def shufflenet_block(tensor, ch, s, g):
```

```
x = gconv(tensor, ch // 4, g)
```

```
x = BatchNormalization()(x)
```

```
x = ReLU()(x)
```

```
x = ch_shuffle(x, g)
```

```
x = DepthwiseConv2D(3, strides=s, padding='same')(x)
```

```
x = BatchNormalization()(x)
```

```
x = gconv(x, ch if s==1 else ch-K.int_shape(tensor)[-1], g)
```

```
x = BatchNormalization()(x)
```

```
if s == 1:
```

```
    x = Add()([tensor, x])
```

```
else:
```

```
    avg = AvgPool2D(3, strides=2, padding='same')(tensor)
```

```
    x = Concatenate()([avg, x])
```

```

    output = ReLU()(x)
    return output

def stage(x, ch, r, g):
    x = shufflenet_block(x, ch, 2, g)

    for i in range(r):
        x = shufflenet_block(x, ch, 1, g)

    return x

input = Input(input_shape)

x = Conv2D(24, 3, strides=2, padding='same')(input)
x = BatchNormalization()(x)
x = ReLU()(x)
x = MaxPool2D(3, strides=2, padding='same')(x)

for ch, r in zip(channels, repetitions):
    x = stage(x, ch, r, g)

x = GlobalAvgPool2D()(x)
output = Dense(n_classes, activation='softmax')(x)

model = Model(input, output)
model.compile(optimizer='Adam', loss='categorical_crossentropy', metrics=['accuracy', tensorflow
.keras.metrics.Precision()])
return model

input_shape = 224, 224, 3
n_classes = 23

K.clear_session()

```

```

model = shufflenet(input_shape, n_classes)
model.summary()
model_path = "model.h5"

from keras.callbacks import ModelCheckpoint

M = ModelCheckpoint(model_path, monitor='accuracy', verbose=1, save_best_only=True)
epochs = 100
batch_size = 128
#### Fitting the model
history = model.fit(
    train_data, steps_per_epoch=train_data.samples // batch_size,
    epochs=epochs,
    validation_data=test_data, validation_steps=test_data.samples // batch_size,
    callbacks=[M])
history.history.keys()
import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(20, 8))
plt.plot(history.history['accuracy'])

for i in range(epochs):
    if i%5 == 0:
        plt.annotate(np.round(history.history['accuracy'][i]*100,2),xy=(i,history.history['accuracy'][i]
))

plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.show()
plt.figure(figsize=(20, 8))
plt.plot(history.history['loss'])

for i in range(epochs):

```

```

        if i%5 == 0:
            plt.annotate(np.round(history.history['loss'][i]*100,2),xy=(i,history.history['loss'][i]))

plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.show()

# Import the necessary libraries.
import numpy as np
import matplotlib.pyplot as plt
import os

# Import the necessary libraries.
import tensorflow as tf
import glob
from PIL import Image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, Dense, MaxPooling2D, Flatten, Convolution2D,
Dropout, BatchNormalization

# Ignoring the warnings
import warnings
warnings.filterwarnings("ignore")

dir_name_train_fresh = 'datasets/train/fresh'
dir_name_train_half_spoiled = 'datasets/train/half_spoiled'
dir_name_train_spoiled = 'datasets/train/spoiled'

def plot_images(item_dir, n=5):
    all_item_dir = os.listdir(item_dir)
    item_files = [os.path.join(item_dir, file) for file in all_item_dir][:n]

    plt.figure(figsize=(35, 10))
    for idx, img_path in enumerate(item_files):
        plt.subplot(2, n, idx+1)
        img = plt.imread(img_path)
        plt.imshow(img, cmap='gray')

```

```

plt.axis('off')

plt.tight_layout()
def Images_details_Print_data(data, path):
    print("==== Images in: ", path)
    for k, v in data.items():
        print("%s:\t%s" % (k, v))

def Images_details(path):
    files = [f for f in glob.glob(path + "**/*.*", recursive=True)]
    data = {}
    data['images_count'] = len(files)
    data['min_width'] = 10**100 # No image will be bigger than that
    data['max_width'] = 0
    data['min_height'] = 10**100 # No image will be bigger than that
    data['max_height'] = 0

    for f in files:
        im = Image.open(f)
        width, height = im.size
        data['min_width'] = min(width, data['min_width'])
        data['max_width'] = max(width, data['max_height'])
        data['min_height'] = min(height, data['min_height'])
        data['max_height'] = max(height, data['max_height'])

    Images_details_Print_data(data, path)
    print("")
    print("Trained data for :")
    print("")

Images_details(dir_name_train_fresh)
print("")
plot_images(dir_name_train_fresh)

```

```

print("")
print("Trained data for :")
print("")

Images_details(dir_name_train_half_spoiled)
print("")
plot_images(dir_name_train_half_spoiled)
print("")
print("Trained data for :")
print("")

Images_details(dir_name_train_spoiled)
print("")
plot_images(dir_name_train_spoiled)

### Data augmentation
train_datagen=ImageDataGenerator(rescale=1./255, shear_range=0.2, zoom_range=0.2,
horizontal_flip=True)
test_datagen=ImageDataGenerator(rescale=1./255)

# having train and test images
training_set=train_datagen.flow_from_directory('datasets/train', batch_size=32,
class_mode='categorical', target_size=(255,255))
testing_set=test_datagen.flow_from_directory('datasets/test', batch_size=32,
class_mode='categorical', target_size=(255,255))

## Creating cnn layer

# initialized the model
model=Sequential()

# Adding first convolution layer
model.add(Convolution2D(32,(3,3),input_shape=(255,255,3), padding='same', activation='relu'))

# Adding first Maxpool layer

```

```

model.add(MaxPooling2D(pool_size=(2,2)))

# Adding Flatten layer
model.add(Flatten())

# Adding input layer
model.add(Dense(units=64, activation='relu', ))

# Adding dropout layer
model.add(Dropout(0.4))

# Adding output layer
model.add(Dense(units=3, activation='softmax'))

model.summary()

# Compiling the model

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
# Fitting the model

history= model.fit(training_set,
                    steps_per_epoch= training_set.samples//32,
                    validation_data=testing_set,
                    validation_steps= testing_set.samples//32,
                    epochs=20)
def plot():
    # Plot training & validation accuracy values

    plt.plot(history.history['accuracy'])
    plt.plot(history.history['val_accuracy'])
    plt.title('Model accuracy')
    plt.ylabel('Accuracy')
    plt.xlabel('Epoch')
    plt.legend(['Train', 'Test'], loc='upper left')

```

```
plt.show()

# Plot training & validation loss values

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Test'], loc='upper left')
plt.show()
plot()
```


PLAGIARISM REPORT

major report SR.docx

ORIGINALITY REPORT

6%

SIMILARITY INDEX

2%

INTERNET SOURCES

2%

PUBLICATIONS

4%

STUDENT PAPERS

PRIMARY SOURCES

1

Submitted to SRM University

Student Paper

2%

2

www.mdpi.com

Internet Source

1%

3

Submitted to University of Technology,
Sydney

Student Paper

<1%

4

www.ijraset.com

Internet Source

<1%

5

Submitted to University of Northumbria at
Newcastle

Student Paper

<1%

6

Submitted to Oxford Brookes University

Student Paper

<1%

7

Prasenjit Dey, Sudip Kumar Adhikari, Sourav
De, Indrajit Kar. "Internet of Things-Based
Machine Learning in Healthcare - Technology
and Applications", CRC Press, 2024

Publication

<1%

8

Submitted to George Washington University

Student Paper

<1%

PUBLICATION

14th INTERNATIONAL CONFERENCE ON
INNOVATIONS IN EDUCATION AND E-LEARNING

HYBRID CONFERENCE
[INPERSON + ONLINE]

27th - 28th March 2025 | **Bangalore, India**

Organized by:
YENEPLOYA
(DEED TO EE UNIVERSITY)
Yenepoya University, Bharatiya City

IFERP Academy
Explore Your Research Journey...

ICRCET | **Scopus**

The CPD Standards Office
CPD PROVIDER: 41182
2024-2026
www.cpdstandards.com
12.2 CPD CREDIT HOURS

Ref No : 78369

Date : 22/04/2025

Conference Secretariat - Chennai, India

Letter of Acceptance

Abstract ID : [ICRCET-2025 BAN 2293](#)

Paper Title : [AI AND IOT SOLUTIONS FOR QUALITY CONTROL IN FRUITS, VEGETABLES, AND MEAT: ENSURING FRESHNESS AND SAFETY](#)

Author Name : [Kirthic Madavan](#),

Co-Author Name : [Raghul Siddarth](#) , [Lavanyah](#) , [Anbazhagu U.V](#)

Institution : [SRM University of science and technology](#)